

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP



Elhan Mahmutovic

Adna Kikanovic

Adnan Musanovic

Mirza Abdulahovic

INTERNATIONAL UNIVERSITY OF SARAJEVO

2026

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP

BY

Elhan Mahmutovic – Software Engineering
Adna Kikanovic – Software Engineering
Adnan Musanovic – Software Engineering
Mirza Abdulahovic – Software Engineering

A graduation project submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science at the
Faculty of Engineering and Natural Sciences
International University of Sarajevo

June 2026

APPROVAL PAGE

I certify that I have supervised and read this study and that in my opinion, it conforms to acceptable standards of scholarly presentation and is fully adequate, in scope and quality, as a graduation project for the degree of Bachelor of Science at the Faculty of Engineering and Natural Sciences.

.....
Assist. Prof. Dr. Ali Abd Almisreb

Mentor

DECLARATION

We hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. We also declare that, as required by these rules and conduct, we have fully cited and referenced all material and results that are not original to this work.

Elhan Mahmutovic

Signature

Adna Kikanovic

Signature

Adnan Musanovic

Signature

Mirza Abdulahovic

Signature

Date

INTERNATIONAL UNIVERSITY OF SARAJEVO

DECLARATION OF COPYRIGHT AND AFFIRMATION OF FAIR USE OF UNPUBLISHED WORK

Copyright © 2026 by Elhan Mahmutovic, Adna Kikanovic, Adnan Musanovic, and Mirza Abdulahovic. All rights reserved.

Quantum-Enhanced Multi-Modal Vector Search: A Hybrid Database Web App

No part of this unpublished work may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without prior written permission of the copyright holder and IUS Library.

Affirmed by:

Elhan Mahmutovic

Signature

Adna Kikanovic

Signature

Adnan Musanovic

Signature

Mirza Abdulahovic

Signature

Date

ACKNOWLEDGEMENTS

[ACKNOWLEDGEMENTS TEXT – see GUIDELINES.md §Acknowledgements]

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CLIP	Contrastive Language – Image Pre-training
CPU	Central Processing Unit
CSWAP	Controlled-SWAP gate
FAISS	Facebook AI Similarity Search
HNSW	Hierarchical Navigable Small World
IBM QPU	IBM Quantum Processing Unit
JSONB	Binary JSON (PostgreSQL data type)
MRR	Mean Reciprocal Rank
NISQ	Noisy Intermediate-Scale Quantum
ORM	Object–Relational Mapping
qRAM	Quantum Random-Access Memory
REST	Representational State Transfer
SIMD	Single Instruction, Multiple Data
SQL	Structured Query Language
ViT	Vision Transformer

TABLE OF CONTENTS

APPROVAL PAGE	iii
DECLARATION	iv
DECLARATION OF COPYRIGHT	v
ACKNOWLEDGEMENTS	vi
LIST OF ABBREVIATIONS	vii
ABSTRACT	x
1 INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement and Research Aims	1
1.3 Research Questions and Objectives	1
2 LITERATURE REVIEW	3
2.1 Classical Vector Search	3
2.2 Cross-Modal Embeddings	3
2.3 Quantum Similarity Primitives	3
2.4 Hybrid Classical–Quantum Architectures	4
3 RESEARCH METHODOLOGY	5
3.1 System Overview	5
3.2 Methodology Diagram	6
3.3 Dataset and Ground Truth	8
3.4 Embedding Pipeline	8
3.5 Search Engines	8
3.5.1 Classical Baselines	8
3.5.2 Quantum Engines	9
3.6 Evaluation Metrics	9
3.7 Experimental Configuration	10

Use of AI Tools	10
4 RESULTS AND ANALYSIS	11
4.1 Accuracy Across Engines	11
4.2 Quantum Resource Cost	11
4.3 Wall-Clock Behaviour (Within-Engine Only)	12
4.4 IBM Hardware Sanity Check	12
4.5 Discussion	12
4.6 Limitations	12
5 SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT	13
6 CONCLUSION	14
REFERENCES	15
A APPENDICES	16
APPENDICES	16

ABSTRACT

QUANTUM-ENHANCED MULTI-MODAL VECTOR SEARCH: A HYBRID DATABASE WEB APP

[ABSTRACT – draft after Results chapter is final.]

Keywords: *quantum computing, vector search, similarity search, swap test, Grover's algorithm, CLIP, benchmarking*

LIST OF TABLES

Table 1:	Benchmark configuration grid.	10
Table 2:	Average Mean Reciprocal Rank by engine. Higher is better. . . .	11

LIST OF FIGURES

Figure 1:	End-to-end pipeline. Data loading and CLIP embedding feed a common engine-strategy interface, behind which the classical, quantum, and hybrid engines are interchangeable. Every run is recorded in PostgreSQL and surfaced through the FastAPI and React interface.	6
Figure 2:	Methodology flow. Four scripts run in order, from dataset import through CLIP indexing, the simulator benchmark sweep, and report generation. The IBM hardware validation is a one-off side arm that writes its single result into the same database table.	7
Figure 3:	Empirical Grover oracle calls vs. the theoretical $\lfloor \pi\sqrt{N}/4 \rfloor$ curve.	11

1. INTRODUCTION

[INTRODUCTION – opening paragraphs framing why cross-modal vector search matters in modern machine-learning systems, and why a side-by-side comparison with quantum similarity is worth doing now even though quantum hardware is not yet competitive. See GUIDELINES.md §Claims-must-be-sourced before writing.]

1.1 Background of Study

[BACKGROUND – define vector search, embeddings, CLIP, and the quantum primitives referenced later (swap test, Grover, amplitude encoding). All claims must point to a reference in `references.bib`.]

1.2 Problem Statement and Research Aims

[PROBLEM STATEMENT – frame the work as an empirical investigation, not a claim of quantum advantage. Borrow framing from docs/midterm/speaker_notes.md slide 3 and slide 11.]

1.3 Research Questions and Objectives

RQ1. *Accuracy*: Does swap-test similarity reproduce the ranking of classical cosine similarity, and what is the shot budget required to do so?

RQ2. *Scaling*: Does our Grover implementation reproduce the $\lfloor \pi\sqrt{N}/4 \rfloor$ oracle-call scaling in practice?

RQ3. *Practicality*: Under what conditions – if any – would a quantum vector-search engine outperform a production classical baseline (HNSW) on the same dataset?

RQ4. *Suitability*: Independent of hardware maturity, is cross-modal vector search a problem class that quantum computing is suited to, given that quantum advantage concentrates on compute-bound problems over small data rather than data-bound retrieval?

[OBJECTIVES – list 3-4 deliverable-shaped goals (engines implemented,

benchmarks collected, web app for inspection, written analysis). Each deliverable must be checkable against the repository.]

2. LITERATURE REVIEW

2.1 Classical Vector Search

Exact nearest-neighbour search at scale was popularised by FAISS, a SIMD-vectorised library for billion-scale similarity search [1]. For datasets where exact search is too expensive, the de facto approximate method is the Hierarchical Navigable Small World graph [2], which trades a small recall loss for $O(\log N)$ retrieval.

[CLASSICAL VECTOR SEARCH – expand the paragraph above; add a brute-force baseline reference if one is wanted.]

2.2 Cross-Modal Embeddings

CLIP aligns a text encoder and an image encoder via contrastive pre-training on 400 million image–caption pairs, producing a shared 512-dimensional space in which cosine similarity captures cross-modal relevance [3].

[CLIP / CROSS-MODAL EMBEDDINGS – expand: explain ViT-B/32, mention that we truncate the 512-dim output to 64/128/256 for benchmarking.]

2.3 Quantum Similarity Primitives

Grover’s algorithm provides a quadratic speedup for unstructured search [4]. The swap test, introduced by Buhrman et al. [5], estimates the squared inner product of two quantum states with a single ancilla measurement. Both algorithms assume that the dataset is already encoded into a quantum state; in the absence of quantum RAM [6], this state preparation step costs $O(N)$ and cancels Grover’s quadratic advantage at the system level [7].

[QUANTUM PRIMITIVES – expand: explain amplitude encoding cost, give the formula for swap-test overlap, foreshadow the qRAM discussion in chapter 4.]

2.4 Hybrid Classical–Quantum Architectures

Quantum machine-learning surveys [8] catalogue hybrid classical–quantum pipelines that delegate the expensive global step to classical hardware and reserve quantum resources for a smaller, structured subproblem. Our `hybrid_hnsw_swap_test` engine follows this pattern: HNSW returns a small candidate pool, then the swap test reranks it on the simulator (implemented with Qiskit [9]).

[HYBRID ARCHITECTURES – expand: discuss other hybrid retrieval approaches in the literature; position our work as an instance of this pattern.]

3. RESEARCH METHODOLOGY

3.1 System Overview

The system is a modular monolith that performs cross-modal retrieval: given a short text query such as “a dog on a beach”, it returns the dataset images whose content best matches that description. Text and images are never compared directly. Instead, the same embedding model maps both into a shared vector space, and retrieval becomes the problem of finding the dataset vectors closest to the query vector. The defining property of the system is that the component performing this closest-vector search is interchangeable. A classical engine, a quantum engine, and a hybrid of the two all expose one common interface, so a single query can be answered by any of them and the rankings placed side by side. This makes the system an instrument for comparing search strategies rather than a single retrieval product.

This interchangeability is enforced by a single design decision that runs through the whole codebase: the strategy pattern. Each stage of the pipeline is defined by an abstract interface, and concrete implementations plug into it. Data loaders implement `BaseDataLoader`, embedding models implement `EmbeddingGenerator`, and every search engine implements `SearchEngineStrategy`, which requires just two operations, `build_index` and `search`. Because all engines satisfy the same contract, the surrounding harness that loads data, generates embeddings, runs queries, and records results is identical for every engine. Any difference we measure between a classical and a quantum engine therefore reflects the engine itself, not the code around it. Which engines run, at which vector dimensions, and with how many measurement shots is selected in a configuration file (`benchmarks.yaml`) rather than by editing source, so an experiment is defined by configuration rather than by code changes.

The pipeline is organised into five phases, shown in Figure 1. In the first phase, data loading, a `DirectoryDataLoader` reads the image files and the accompanying ground-truth pairs that define which image each query should retrieve. In the second phase, the embedding pipeline, a CLIP model converts every image and every text query into a vector. In the third phase the search engines build their indexes over these vectors and answer queries, each engine applying its own strategy, whether classical, quantum, or hybrid. In the fourth phase the outcome of every run, including its accuracy and resource measurements, is written to a PostgreSQL database. The fifth phase exposes

the system through a FastAPI backend and a React interface, which lets a user issue a query and inspect the rankings returned by each engine. The sections that follow describe each phase in turn: the dataset and ground truth in Section 3.3, the embedding pipeline in Section 3.4, and the individual search engines in Section 3.5.

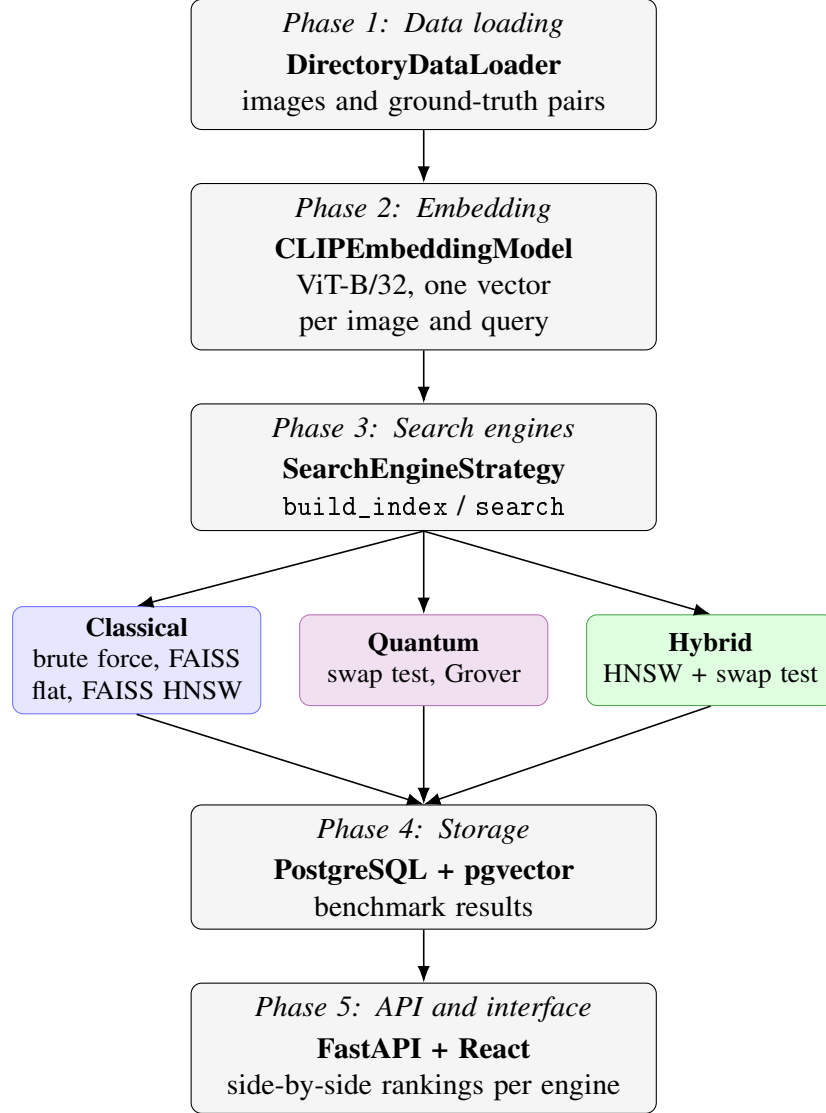


Figure 1: End-to-end pipeline. Data loading and CLIP embedding feed a common engine-strategy interface, behind which the classical, quantum, and hybrid engines are interchangeable. Every run is recorded in PostgreSQL and surfaced through the FastAPI and React interface.

3.2 Methodology Diagram

The study follows a fixed sequence of steps, shown in Figure 2, each implemented by a single script so the whole experiment can be reproduced from a clean checkout. The first two steps prepare the data. The script `import_dataset.py` downloads a fixed subset of the Flickr30k dataset and, in the same pass, writes the ground-truth file that

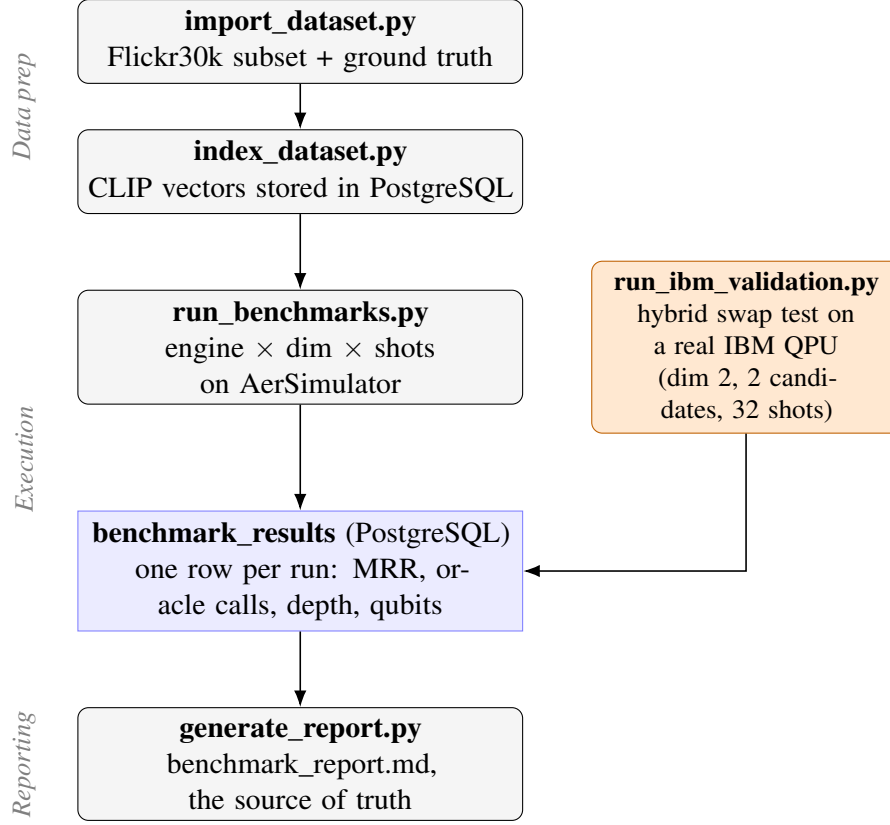


Figure 2: Methodology flow. Four scripts run in order, from dataset import through CLIP indexing, the simulator benchmark sweep, and report generation. The IBM hardware validation is a one-off side arm that writes its single result into the same database table.

records which image each text query should retrieve. This download is deterministic: every run regenerates the same image set and the same query-to-image pairing, so the results do not drift between machines. The second script, `index_dataset.py`, loads each image, passes it through the CLIP model to obtain a vector, and stores that vector in PostgreSQL. After these two steps the database holds one embedding per image, ready to be searched.

With the embeddings in place, `run_benchmarks.py` carries out the main experiment. It iterates over every combination of engine, vector dimension, and shot count listed in `benchmarks.yaml`, and for each combination it answers all of the ground-truth queries. The quantum and hybrid engines execute their circuits on Qiskit’s `AerSimulator`, a noiseless simulator that isolates the effect of finite sampling (shot noise) from the physical noise of real hardware. Every individual run is recorded as one row in the `benchmark_results` table, together with its accuracy and its resource cost: the Mean Reciprocal Rank of the retrieved images, and, for the quantum engines, the number of oracle calls, the circuit depth, and the qubit count. Because the configuration file drives the sweep, widening the experiment to a new dimension or shot count is a

change to that file alone, not to the code.

Alongside the simulator sweep, `run_ibm_validation.py` runs the hybrid swap-test engine once on real IBM quantum hardware, at the smallest workable size: two dimensions, two candidate images, and 32 shots. Its purpose is narrow, to confirm that the swap-test circuit executes on a physical device and returns a sensible result, rather than to measure scaling. The single outcome is written to the same `benchmark_results` table, tagged as the `hybrid_hnsw_swap_test_ibm` engine, so it sits beside the simulator runs without being confused for one.

The final step turns the stored runs back into something readable. The script `generate_report.py` queries the `benchmark_results` table and produces a Markdown summary, `benchmark_report.md`, that groups the runs by engine and reports accuracy per dimension and per shot count. This report is the single source of truth for the Results chapter: every figure quoted in Chapter 4 traces back to a row this script read from the database, rather than to any number recorded by hand. Keeping the analysis in one generated file, derived directly from the stored runs, is what lets the whole study be regenerated by re-running the four scripts in order.

3.3 Dataset and Ground Truth

[DATASET – source, size, how queries were derived, what counts as a correct retrieval. Reference backend/data/ground_truth.jsonc.]

3.4 Embedding Pipeline

[EMBEDDINGS – model, normalisation, truncation rationale.]

3.5 Search Engines

3.5.1 Classical Baselines

Brute-Force Cosine

[BRUTE FORCE – NumPy dot product against every vector. Ground truth.]

FAISS Flat (Exact)

[FAISS FLAT – SIMD-vectorised exact search.]

FAISS HNSW (Approximate)

[FAISS HNSW – multi-layer proximity graph, $O(\log N)$ retrieval.]

3.5.2 Quantum Engines

Qiskit Swap Test

The swap test estimates the squared inner product of two amplitude-encoded vectors $|\psi\rangle$ and $|\phi\rangle$ via a single ancilla measurement:

$$P(\text{ancilla} = 0) = \frac{1 + |\langle\psi|\phi\rangle|^2}{2}. \quad (1)$$

[SWAP TEST – explain circuit (Hadamard, controlled-SWAP, Hadamard, measure), shot-noise scaling, and the choice of AerSimulator over IBM hardware.]

Qiskit Grover (Hardcoded Oracle)

[GROVER – $\lfloor \pi\sqrt{N}/4 \rfloor$ oracle calls, how we measure that empirically, why the oracle is classically pre-specified (qRAM caveat).]

Qiskit Grover with Quantum State Preparation

[GROVER (quantum prep) – contrast with the hardcoded variant.]

Hybrid HNSW + Swap Test

[HYBRID – HNSW prefilter, swap-test rerank. Discuss the rationale.]

IBM Hardware Validation

[IBM HARDWARE – the dimension-2, 2-candidate, 32-shot run on real QPU. Frame as a sanity check, not a scaling study.]

3.6 Evaluation Metrics

[MRR DEFINITION – formula + intuition.]

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (2)$$

where rank_q is the position of the first relevant result for query $q \in Q$ and $1/\text{rank}_q := 0$ if no relevant result is in the top- k .

[ORACLE CALLS, CIRCUIT DEPTH, QUBIT COUNT – define each, state which engines populate which fields.]

3.7 Experimental Configuration

Table 1: Benchmark configuration grid.

Parameter	Values
Engines	[PULL FROM benchmarks.yaml]
Dimensions	64, 128
Shots	512, 1024
Layers	1
Top- k	10

Use of AI Tools

[USE OF AI TOOLS – list every AI-assisted activity: code generation / review, prose drafting, diagram drafting, brainstorming. State explicitly which parts of the algorithm design, implementation, and analysis are the authors’ own. Responsibility for correctness rests with the authors. See GUIDELINES.md §AI Disclosure for the exact phrasing template.]

4. RESULTS AND ANALYSIS

[OPENING SUMMARY – one paragraph stating the headline findings: (1) which engine ranks first on MRR, (2) whether swap test matches classical ranking, (3) whether Grover hits its theoretical oracle-call count, (4) the practical position of the hybrid engine. All numbers must come from the benchmark DB.]

4.1 Accuracy Across Engines

Table 2: Average Mean Reciprocal Rank by engine. Higher is better.

Engine	Dim	Shots	Avg MRR	Runs
<i>Fill from /api/benchmarks or scripts/generate_report.py.</i>				

[MRR DISCUSSION – comment on which engines tied, which dropped, and what the shot/dimension interaction looks like.]

4.2 Quantum Resource Cost

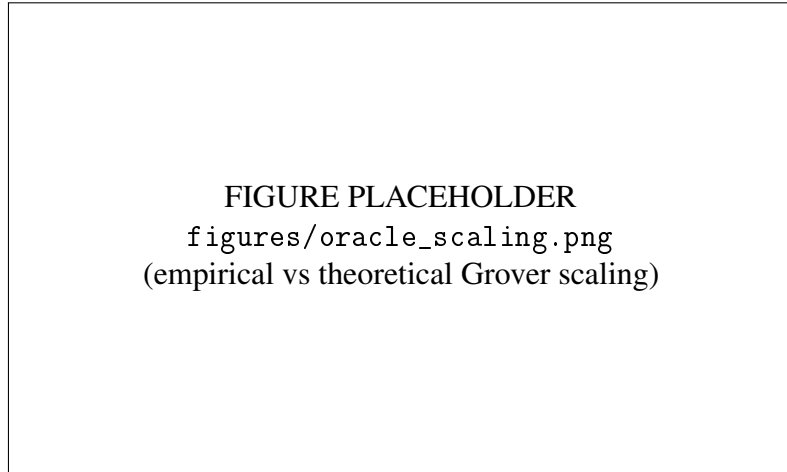


Figure 3: Empirical Grover oracle calls vs. the theoretical $\lfloor \pi\sqrt{N}/4 \rfloor$ curve.

[RESOURCE COST DISCUSSION – qubits per dimension, circuit depth growth, how it matches the theory.]

4.3 Wall-Clock Behaviour (Within-Engine Only)

[WALL-TIME DISCUSSION – restrict comparisons to within-engine across dimensions. Explicitly state why cross-engine wall-time is misleading.]

4.4 IBM Hardware Sanity Check

[IBM SANITY CHECK – results of the validation run, what we learned, what we did not (and cannot) measure from it.]

4.5 Discussion

[DISCUSSION – one short paragraph per research question, each ending with a one-sentence answer.]

4.6 Limitations

[LIMITATIONS – list, briefly, every constraint a reviewer might point to. Pre-empting limitations is stronger than letting the committee find them.]

5. SUSTAINABILITY, INCLUSIVITY, AND SOCIETAL IMPACT

[SUSTAINABILITY & IMPACT – address: (1) energy / compute efficiency of the benchmarking workflow (we deliberately ran on simulator + small dataset rather than burning GPU hours), (2) data privacy (Flickr30k is public, no PII), (3) educational impact (the codebase and benchmarks are reproducible and openly available), (4) honest framing of quantum capabilities as a counterweight to hype. Map loosely to SDG 4 (quality education) and SDG 9 (industry, innovation, infrastructure) if that aids the bonus. See GUIDELINES.md §Sustainability for tone notes.]

6. CONCLUSION

[SUMMARY – restate the three research questions and their answers in two to three sentences.]

[CONTRIBUTIONS – list what this work delivered: (1) the benchmark infrastructure, (2) the empirical verification of swap-test and Grover behaviour, (3) the web application for inspection, (4) the honest analysis of where the quantum-classical gap actually lies. The honest framing IS a contribution - say so.]

[FUTURE WORK – candidates: larger dataset (Flickr30k full set or ImageNet subset), wider dimension sweep (256, 512), proper variational ansatz study, comparison against pgvector HNSW as a third classical baseline, IBM hardware noise characterisation.]

REFERENCES

- [1] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [3] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [4] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [5] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, “Quantum fingerprinting,” *Physical Review Letters*, vol. 87, no. 16, p. 167902, 2001.
- [6] V. Giovannetti, S. Lloyd, and L. Maccone, “Quantum random access memory,” *Physical Review Letters*, vol. 100, no. 16, p. 160501, 2008.
- [7] S. Aaronson, “Read the fine print,” *Nature Physics*, vol. 11, no. 4, pp. 291–293, 2015.
- [8] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.
- [9] Qiskit Contributors, “Qiskit: An open-source framework for quantum computing,” 2024, <https://qiskit.org/>.

A. APPENDICES

Appendix A: Engine Configuration File

[Reproduce backend/config/benchmarks.yaml verbatim with a `lstlisting` block.]

Appendix B: Database Schema

[Reproduce the benchmark_results table definition from db/migrations.]

Appendix C: Repository

[Link to the public GitHub repository and tag of the report-version commit.]